COMP3620 Gradescope Exercise One

Date set: 7 October 2026 **Date due:** 21 October 2026 (23:59)
This exercise contributes 10% to the final mark.

Task: You will implement the perceptron algorithm as presented in the worksheet of week 1. You do not need to provide a learning rate, or standardise the input. You may at any time ask one of the TAs for advice.

Your solution is tested by an autograder. It has the Python standard library, NumPy, pandas and matplotlib available; you need only the standard library and NumPy. PyTorch, scikit-learn, TensorFlow, Keras and JAX are not allowed. Any other library is not installed. The autograder checks your imports first and tells you by name if one is not allowed or not installed; in that case your code is not run and no marks are given.

Perceptron practical — submission instructions

Submit a single file named `solution.py` that defines the two functions below. The data file `stroke_separable.csv` has three columns: `age_years`, `blood_pressure_mmhg`, `stroke` (the class label, 0 or 1).

1. `load_data(filename)` — 3 marks

Read the CSV and return a tuple of three sequences:

```
age, hg, cls = load_data("stroke_separable.csv")
```

- `age` — the `age_years` column
- `hg` — the `blood_pressure_mmhg` column
- `cls` — the `stroke` column, as the **raw 0/1 values from the file** (do **not** convert the labels here)

Return them in the order `(age, hg, cls)`. Remember to skip the header row.

2. `perceptron(age, hg, cls)` — 7 marks

Inside this function run the perceptron algorithm and return the three weights of the perceptron

```
y = w0 + w_age * age + w_hg * hg
```

as a tuple **in this order**:

```
w0, w_age, w_hg = perceptron(age, hg, cls)
```

Your function must:

- **set a random seed** (so your run is reproducible);
- **stop**

  You can, for example, set a large value of `N`, the number of steps, inside the *perceptron* function, once you've verified that it produces a correct solution. You are free to design other stopping criteria, but your algorithm must stop and when it does, your perceptron must separate the classes perfectly.

Internally, you may also **convert the 0/1 labels if this leads to a simpler version of the algorithm** — but this is your choice, not a requirement. Any perceptron that returns weights separating the data is acceptable, whatever label convention it uses internally.

The grader calls `perceptron(age, hg, cls)` with the output of your own `load_data`, and with no N argument. A mistake in `load_data` will also cost you the marks for `perceptron`.

How the weights are graded

Full marks if your line **perfectly separates the two classes**. A correct line with the classification **inverted** (positive/negative side swapped) also receives full marks — only the separation matters, not the sign convention. Specific weight values are **not** checked (the algorithm is stochastic), only whether they classify every point correctly.

Use of AI or library implementations

We ask you to write the implementation yourself and not to submit AI-generated code or use library implementations, e.g. provided by PyTorch. In the first instance we will inspect solutions using an autograder, but we may look into the code to see if a proper implementation has been attempted.